

Chapter 6: Collections

This lesson covers collections. Collections are a means of temporarily storing the data associated with an application. In this lesson we will take a look at arrays, structures and enums. Arrays and enums are still used regularly, structures have largely been replaced by classes which we will tackle in our next couple of chapters. Again, I include the structures syntax for those of you who might be taking a certification test.

Okay, let's jump to the inside and take a look at our collections syntax.

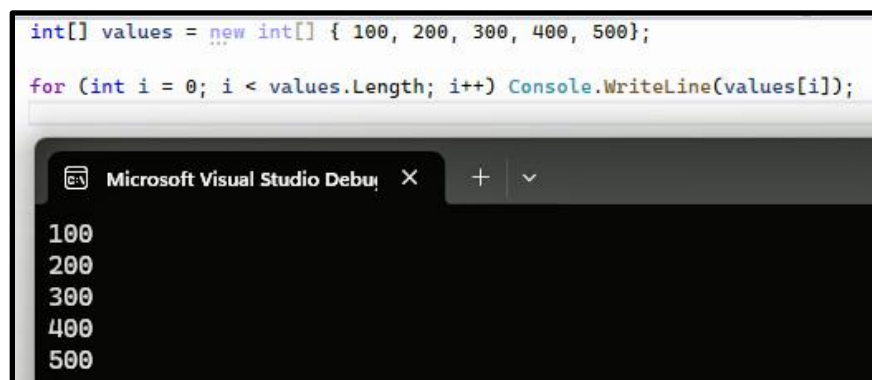
Please begin by creating a new console application named chapter 6 collections.

Go ahead and create the code for our first example (Figure 6.1). This example demonstrates the use of a one-dimensional array. In our example here we tell the compiler that we want to declare a one-dimensional array of integers with the `int[]` syntax. We then create an instance of that array type with the `values = new int[]` syntax. Finally, we populate that array with the `{100, 200, 300, 400, 500}` syntax. We then of course end the command with a semicolon.

In our last chapter we learned about the for loop so that syntax should be familiar. The use of the `values.Length` method is new but pretty self-explanatory. Remember, we don't need curly brackets with this code block because it contains only a single command.

Okay, nice little array example to get us started.

Figure 6.1: One-Dimensional Array



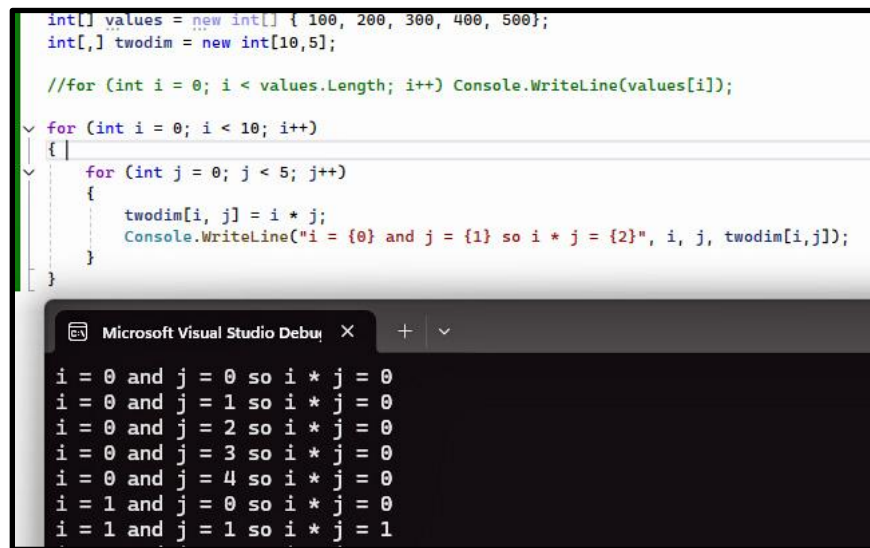
```
int[] values = new int[] { 100, 200, 300, 400, 500};  
for (int i = 0; i < values.Length; i++) Console.WriteLine(values[i]);
```

Microsoft Visual Studio Debug Console

100
200
300
400
500

In our next example we play with a two-dimensional array. Here is the deal with arrays, they can have as many dimensions as you want, but the data type of each cell must be the same. In our example here each element in our two-dimensional array must be of type integer.

Figure 6.2: Two-Dimensional Array



```
int[] values = new int[] { 100, 200, 300, 400, 500 };
int[,] twodim = new int[10,5];

//for (int i = 0; i < values.Length; i++) Console.WriteLine(values[i]);

for (int i = 0; i < 10; i++)
{
    for (int j = 0; j < 5; j++)
    {
        twodim[i, j] = i * j;
        Console.WriteLine("i = {0} and j = {1} so i * j = {2}", i, j, twodim[i,j]);
    }
}
```

Microsoft Visual Studio Debug Console Output:

```
i = 0 and j = 0 so i * j = 0
i = 0 and j = 1 so i * j = 0
i = 0 and j = 2 so i * j = 0
i = 0 and j = 3 so i * j = 0
i = 0 and j = 4 so i * j = 0
i = 1 and j = 0 so i * j = 0
i = 1 and j = 1 so i * j = 1
```

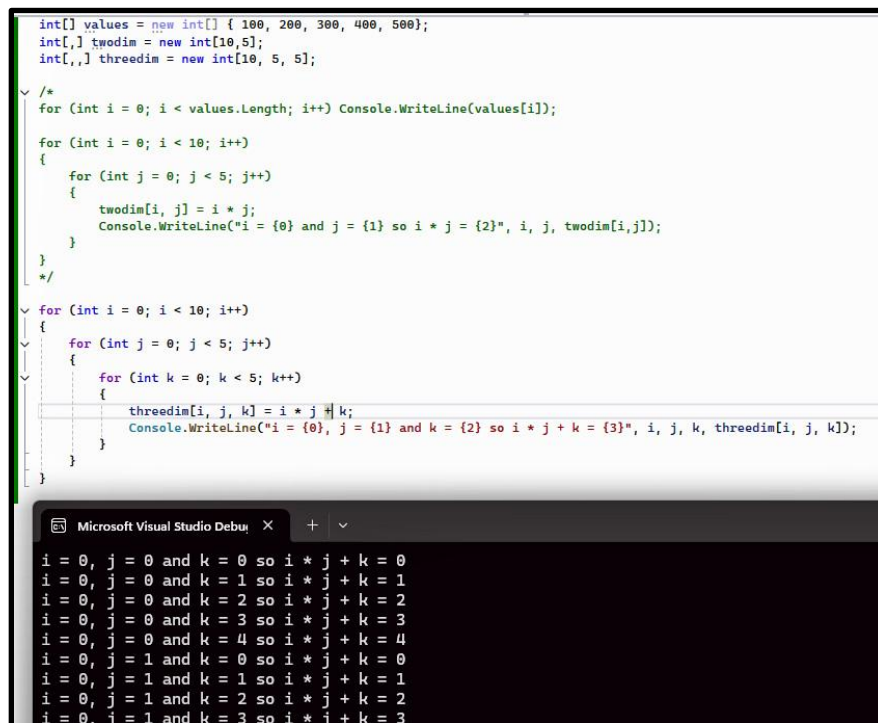
So, what did this example teach us?

- We learned how to declare, instantiate and populate a two-dimensional array. Pretty straight forward once you see the syntax.
- Next, we learned you could nest for loops as long as their loop control variables are different. In our example here we used the variable *i* for the outer for loop control variable and the variable *j* as our inner for loop control variable. Very common method for walking through all the elements of a two-dimensional array.
- Remember, C# arrays are zero-indexed.

In our next example we nest our for loops three deep to walk through all the elements of a three-dimensional array. (Figure 6.3) You of course are now asking how deep can we nest for loops and how many dimensions can an array have? The short answer is as many as you want. Be careful however, because your brain may be able to handle a 10-dimensional array, but mine certainly can't. I get much past 3 or 4 and my humble brain quickly runs out of gas.

Did you notice how I had to specify the number of elements in each of our array dimensions. That is because I did not populate the array in that same command. The compiler needs to know how much memory to allocate for the array, without it being populated you need to tell it how much memory you are going to need. In our case 10 elements in the first dimension and five elements in each of the second and third dimensions.

Figure 6.3: Three-Dimensional Array



```
int[] values = new int[] { 100, 200, 300, 400, 500};
int[,] twodim = new int[10,5];
int[,,] threedim = new int[10, 5, 5];

/*
for (int i = 0; i < values.Length; i++) Console.WriteLine(values[i]);

for (int i = 0; i < 10; i++)
{
    for (int j = 0; j < 5; j++)
    {
        twodim[i, j] = i * j;
        Console.WriteLine("i = {0} and j = {1} so i * j = {2}", i, j, twodim[i,j]);
    }
}
*/

for (int i = 0; i < 10; i++)
{
    for (int j = 0; j < 5; j++)
    {
        for (int k = 0; k < 5; k++)
        {
            threedim[i, j, k] = i * j * k;
            Console.WriteLine("i = {0}, j = {1} and k = {2} so i * j * k = {3}", i, j, k, threedim[i, j, k]);
        }
    }
}
```

Microsoft Visual Studio Debug Console Output:

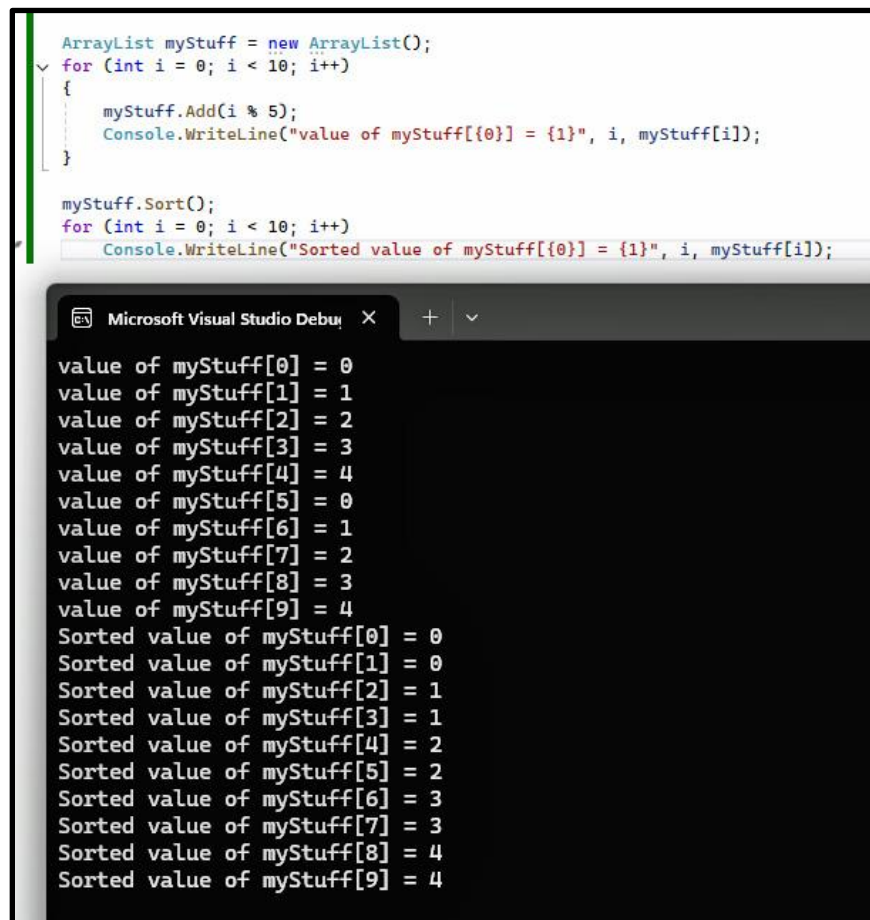
```
i = 0, j = 0 and k = 0 so i * j * k = 0
i = 0, j = 0 and k = 1 so i * j * k = 1
i = 0, j = 0 and k = 2 so i * j * k = 2
i = 0, j = 0 and k = 3 so i * j * k = 3
i = 0, j = 0 and k = 4 so i * j * k = 4
i = 0, j = 1 and k = 0 so i * j * k = 0
i = 0, j = 1 and k = 1 so i * j * k = 1
i = 0, j = 1 and k = 2 so i * j * k = 2
i = 0, j = 1 and k = 3 so i * j * k = 3
```

In our next example we introduce an ArrayList. The ArrayList class differs from a standard array object in that memory may be dynamically allocated. That is to say ArrayLists allow for the addition and deletion of memory during execution. (Figure 6.4)

So, what warrants our attention in this example?

- First, the using System.Collection syntax. This command makes available the System.Collections namespace. We'll talk about namespaces in Chapter 9. For now, just know that it makes a bunch of Microsoft-provided classes and their associated methods available to us in this routine.
- Next, please note how we declare an instance of a Microsoft-provided class. In our case the ArrayList class. ArrayList (which is the class name) myStuff (which is the instance name) = new (keyword) ArrayList() (constructor – which is just a method of a class which creates an instance of itself).
- Next, please note how we used the modulo operator to assign values to our ArrayList.
- Come on, the Sort method of the ArrayList class looks pretty cool. Again, provided by Microsoft for your use.

Figure 6.4: ArrayList Class



```
ArrayList myStuff = new ArrayList();
for (int i = 0; i < 10; i++)
{
    myStuff.Add(i % 5);
    Console.WriteLine("value of myStuff[{0}] = {1}", i, myStuff[i]);
}

myStuff.Sort();
for (int i = 0; i < 10; i++)
    Console.WriteLine("Sorted value of myStuff[{0}] = {1}", i, myStuff[i]);
```

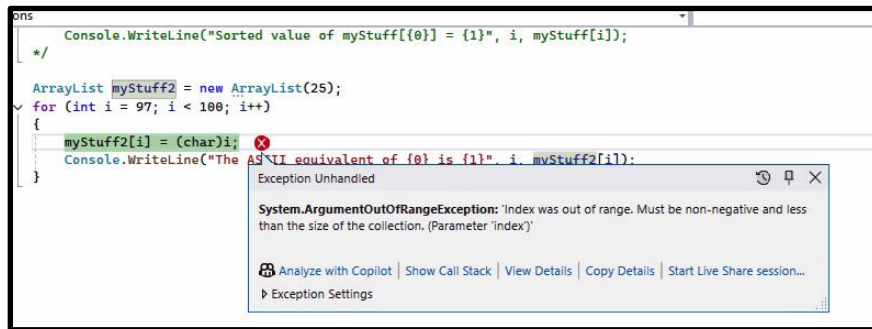
Microsoft Visual Studio Debug Console

```
value of myStuff[0] = 0
value of myStuff[1] = 1
value of myStuff[2] = 2
value of myStuff[3] = 3
value of myStuff[4] = 4
value of myStuff[5] = 0
value of myStuff[6] = 1
value of myStuff[7] = 2
value of myStuff[8] = 3
value of myStuff[9] = 4
Sorted value of myStuff[0] = 0
Sorted value of myStuff[1] = 0
Sorted value of myStuff[2] = 1
Sorted value of myStuff[3] = 1
Sorted value of myStuff[4] = 2
Sorted value of myStuff[5] = 2
Sorted value of myStuff[6] = 3
Sorted value of myStuff[7] = 3
Sorted value of myStuff[8] = 4
Sorted value of myStuff[9] = 4
```

The biggest difference between the use of an array and an ArrayList is that the ArrayList class does not support multiple dimensions. I never used them due to this limitation. If I couldn't populate a multi-dimensional array initially, I would simply set the number of elements in each dimension overly large such that I never exceeded allocated memory space. Wasted memory may have been a big deal 20 years ago, but today memory is so cheap it is absolutely no big deal.

Why is it important to allocate enough memory for your arrays? Because if your code references an array element without an associated memory allocation you get a runtime error. Your program stops right in the middle of execution. Most definitely not cool. (Figure 6.5)

Figure 6.5: Runtime Error



We will study program errors in detail in a subsequent lesson, but let me say now that there are three main types of bugs or programmatic errors: syntax errors, logic errors, and runtime errors.

1. Syntax errors are the easy ones because intellisense is going to recognize them as you type and underline them in red. You see red underlines in your code you know that you have syntax errors. The compiler will not play with syntactically flawed code. I'm sure you have encountered a number of syntax errors as you have been following along with these examples so far.
2. Logic errors are true bugs. Your code compiles properly but does not behave as you intended. Logic errors are the bane of all professional programmers. "Why is this not working?" is the typical refrain. Worse is when those logic errors get past the testing of your application and end up in front of users. We have all had it happen to us as a user, "why is this thing not working ... crappy website, I am going somewhere else".
3. Runtime errors are the most frustrating for the user and by far the most embarrassing for the programmer. Imagine the user working along when all of the sudden a very obscure error message comes up on their screen and the application just halts. Some poor customer service person is about to get a very angry call after which some poor programmer is going to receive a stern lecture.

More on error handling and the testing of your applications as we move forward. For now, let's take a look at this next example wherein we introduce the struct keyword. (Figure 6.6) There is a lot to digest here so let's break it down and examine each piece in turn.

Figure 6.6: struct keyword

```
Person me = new Person();
Person myWife = new Person();

me.getValues(0, "Rick", "Phillips", 68, "Male");
myWife.getValues(1, "Mary", "Phillips", 67, "Female");

me.displayValues();
myWife.displayValues();

4 references
struct Person
{
    private int i, a;
    private string f, l, g;

    2 references
    public void getValues(int ID, string fname, string lname, int age, string gender)
    {
        i = ID;
        f = fname;
        l = lname;
        a = age;
        g = gender;
    }

    2 references
    public void displayValues()
    {
        Console.WriteLine("{0} {1} is ID {2}, age = {3} and gender = {4}", f, l, i, a, g);
    }
}
```

Microsoft Visual Studio Debug Console

Rick Phillips is ID 0, age = 68 and gender = Male
Mary Phillips is ID 1, age = 67 and gender = Female

- First, note my use of the object named Person and the syntax `Person me = new Person();` This syntax says create an instance of an object of type Person called me using the constructor method named `Person()`. Okay, this is your first programming standard, because I don't really care if you ever use PascalCase, but this one should always be used. The method which creates an instance of an object or class in memory should always be named the exact same as the object or class it creates. In our case, Person is the object name and `Person()` is the constructor method which creates an instance of the Person object in memory. Remember, C# is case sensitive, so the uppercase P is important.
- Next, check out the keyword `struct`. Do you suppose that is short for structure? Well sure it is. Structures existed in C long before classes and object-oriented programming. They still exist, but have been largely replaced by their class counterparts. Regardless, this syntax says I want to define a structure named Person with an uppercase P. I will be using five private variables which means they are only available within the scope of this structure. I will also create two methods on my Person object. One is called `getValues` and the second is called `displayValues`.

- I think the output shown at the bottom of the figure pretty much explains how this little routine works. But there is one caveat to remember, the struct definition has to come after the declaration and any calls to the structure methods. C# has a two-pass compiler so the calls to an object must precede the object's definition.
- One more point to consider. There is no method named Person to be used as a constructor in this example. That is because the compiler knows that the constructor needs to exist and creates it behind the scenes for you - thanks compiler.

In our next example I start to combine some of our collection concepts by creating an array aptly named myArray with two elements each of type Person. (Figure 6.7) Please study this example until you can honestly say in your head "Yeah, I get it, this syntax is starting to make sense..."

Figure 6.7: array of struct

```

Person me = new Person();
Person myWife = new Person();
Person[] myArray = new Person[2]; // create a 1-dimensional array of the Person structure

me.getValues(0, "Rick", "Phillips", 68, "Male");
myWife.getValues(1, "Mary", "Phillips", 67, "Female");

myArray[0] = me;
myArray[1] = myWife;

//me.displayValues();
//myWife.displayValues();

myArray[0].displayValues();
myArray[1].displayValues();

```

```

6 references
struct Person
{
    private int i, a;
    private string f, l, g;

    2 references
    public void getValues(int ID, string fname, string lname, int age, string gender)
    {
        i = ID;
        f = fname;
        l = lname;
        a = age;
        g = gender;
    }

    2 references
    public void displayValues()
    {
        Console.WriteLine("{0} {1} is ID {2}, age = {3} and gender = {4}", f, l, i, a, g);
    }
}

```

```

Microsoft Visual Studio Debug Console
Rick Phillips is ID 0, age = 68 and gender = Male
Mary Phillips is ID 1, age = 67 and gender = Female

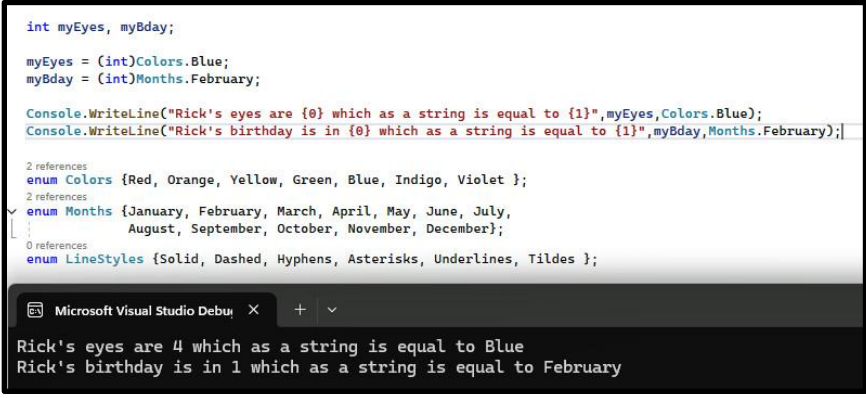
```


We conclude this lesson with an enum example. Enum is short for enumeration and all it means is that we give each string element in a set a corresponding integer value and then display the string while storing the integer. Microsoft uses enums to reference colors, line styles and anything that has a long list of string options. The point is to minimize the use of memory. When enums were first invented memory was still very expensive and storing the integer values not only saved on memory cost but also helped to scale the application as integer storage is much more efficient than the storage of strings.

Our enum example is very simple, but I'm sure you get the idea. (Figure 6.8) Define an enum type and give it a name, then provide a list of all the string elements. The compiler will assign the first element in the set a value of 0 and the second element in the set a value of 1, etc. Super easy.

One thing about our enum example does warrant discussion. The `myEyes = (int)Colors.Blue;` syntax says take the `Colors.Blue` string and cast it to its integer equivalent. Casting basically means to take one data type and convert it to a second type, either primitive or user-defined.

Enum Figure 6.8: enum



```
int myEyes, myBday;
myEyes = (int)Colors.Blue;
myBday = (int)Months.February;

Console.WriteLine("Rick's eyes are {0} which as a string is equal to {1}", myEyes, Colors.Blue);
Console.WriteLine("Rick's birthday is in {0} which as a string is equal to {1}", myBday, Months.February);

2 references
enum Colors {Red, Orange, Yellow, Green, Blue, Indigo, Violet };
2 references
enum Months {January, February, March, April, May, June, July,
August, September, October, November, December};
0 references
enum LineStyles {Solid, Dashed, Hyphens, Asterisks, Underlines, Tildes };
```

Microsoft Visual Studio Debug Console

```
Rick's eyes are 4 which as a string is equal to Blue
Rick's birthday is in 1 which as a string is equal to February
```

This concludes our discussion of collections. Let's jump to the outside and review what we learned.

- We first learned about arrays. Couple of things to remember about arrays. First, all the elements of an array must be of the same data type, whether a data type primitive or user-defined. Remember that all arrays are zero-indexed.
- We next took a look at multi-dimensional arrays and how we could walk through the elements of those arrays with nested for loops. Very common practice.
- The `ArrayList` class came next. Nice in that you could dynamically allocate memory but limited to a single dimension so not terribly useful. Study it if you are taking a certification test otherwise forget about it.

- We next moved onto the struct keyword. Very handy means of storing diverse data types in a single object back in the day, but probably now fully replaced by classes which we will introduce in the our next chapter. Again, if you are taking a test this is important otherwise feel free to forget about it.
- We concluded with the enum class. Very handy way of storing a long list of strings in less, more efficient, memory. Use them regularly.

Okay, this concludes our lesson on collections. In our next two lessons we will study classes, the heart of object-oriented programming. Should be great fun.